

Exploring Deep Learning Techniques for Image Reconstruction in CT Scans with External Radiation Exposure: A Case Study in Dose Prediction for Prostate Cancer Patients

Supervisor: Perre-Emmanuel Leni

Prepared by: Ali Fadlallah

Master CompuPhys, EIPHI Graduate School, Université de Bourgogne-Franche-Comté
`ali.fadlallah@edu.univ-fcomte.fr`

June 24, 2023

Contents

1	Introduction	3
2	Background	4
2.1	Literature Review	4
2.2	Computed Tomography (CT) Imaging	4
2.3	Deep Learning and Medical Image Analysis	5
3	Data	6
3.1	Clinical Setup	6
3.2	Dataset	7
3.3	Data Preprocessing ¹	7
3.3.1	CT Scan Preprocessing	7
3.3.2	Fluence Data Preprocessing	7
4	Model Training and Evaluation	9
4.1	UNet Architecture	9
4.2	Training	10
4.2.1	Dataset Preparation	10
4.2.2	Model Compilation	10
4.2.3	Model Training	10
4.2.4	Model Evaluation	11
4.2.5	Saving Images	11
4.3	Model Output and Logs	12
5	Results	12
5.1	Training and Evaluation	12
5.2	Reconstructed Images	14
5.3	Terminal Output	14
5.4	Output Analysis	14
6	Discussion	15
6.1	Analysis of Results	15
6.2	Potential Solutions	15
7	Challenges	15
8	Conclusion	16
A	Appendix	18
A.1	Preprocessing Method for Raw Data	18
B	Technical Report	19
B.1	Code Structure	19
B.2	Model Definition	19
B.3	Modules	21
B.4	Usage	21

Abstract

This research project explores the potential of deep learning techniques in image reconstruction for CT scans with external radiation exposure, specifically aiming to predict the dose distribution for prostate cancer patients. The objective is to develop an accurate and efficient deep learning model to replace the current Monte Carlo simulation method used for dose prediction in radiation therapy. The project involves constructing a deep learning model using TensorFlow and training it on a high-quality dataset of CT scan images. However, the achieved results did not meet the desired accuracy and image quality. The limitations primarily stem from suboptimal model architecture and parameter choices. By identifying these limitations, this research provides valuable insights into improving the accuracy and effectiveness of dose prediction for prostate cancer patients. Future research should focus on refining the model's architecture and optimizing parameter configurations to enhance the quality and efficiency of dose prediction in radiation therapy.

Keywords: deep learning, image reconstruction, CT scans, external radiation exposure, dose prediction, prostate cancer, Monte Carlo simulation, TensorFlow.

1 Introduction

Radiation therapy is a widely used treatment modality for cancer patients, aiming to deliver precise and targeted doses of radiation to tumor sites while minimizing damage to surrounding healthy tissues. Accurate dose prediction plays a crucial role in radiation therapy treatment planning, as it ensures effective tumor control and reduces the risk of complications. Traditional dose prediction methods, such as Monte Carlo simulations, involve computationally intensive processes that can be time-consuming and impractical for real-time clinical applications.

In recent years, deep learning techniques have shown remarkable success in various medical imaging tasks, including image segmentation and classification [2]. These techniques have the potential to revolutionize dose prediction in radiation therapy by providing faster and more accurate predictions. One such deep learning architecture, the U-Net, has gained significant attention due to its ability to capture fine details and spatial relationships in medical images [4].

The objective of this study is to explore the potential of deep learning in image reconstruction for computed tomography (CT) scans with external radiation exposure. Specifically, we aim to develop a U-Net model that can accurately predict the dose distribution for prostate cancer patients based on CT scan images. By training the U-Net model on a dataset of CT scan images and corresponding dose distributions, we can generate binary images representing the predicted dose distribution.

This report presents the methodology, results, and analysis of the developed U-Net model for dose prediction. We evaluate the performance of the model using quantitative metrics and compare it with the ground truth dose distributions obtained from Monte Carlo simulations. Furthermore, we discuss the potential implications of incorporating deep learning-based dose prediction into radiation therapy treatment planning.

The rest of the report is organized as follows. Section 2 provides a literature review of related work in the field of deep learning for medical image analysis and dose prediction. Section 3 describes the methodology, including the dataset, preprocessing steps, and the architecture of the U-Net model. Section 4 presents the experimental results and performance evaluation. In Section 5, we discuss the implications and potential benefits of deep learning-based dose prediction. Finally, Section 6 concludes the report and provides directions for future research.

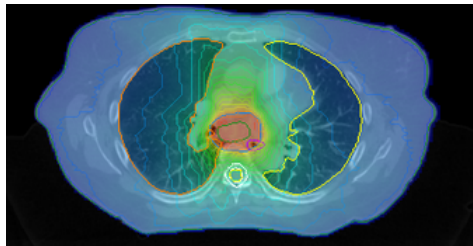


Figure 1: Illustration of a dose distribution in colorwash (blue: low to red: high) on a CT-Slice

2 Background

2.1 Literature Review

Deep learning algorithms, particularly convolutional networks, have emerged as a powerful methodology for analyzing medical images. Litjens et al. [2] conducted a comprehensive survey on deep learning in medical image analysis. The authors reviewed over 300 contributions in the field, focusing on image classification, object detection, segmentation, registration, and other related tasks. The survey covered various application areas, including neuro, retinal, pulmonary, digital pathology, breast, cardiac, abdominal, and musculoskeletal imaging. It provided concise overviews of the studies in each area and discussed the current state-of-the-art, open challenges, and directions for future research. This survey serves as a valuable resource for understanding the landscape of deep learning in medical image analysis.

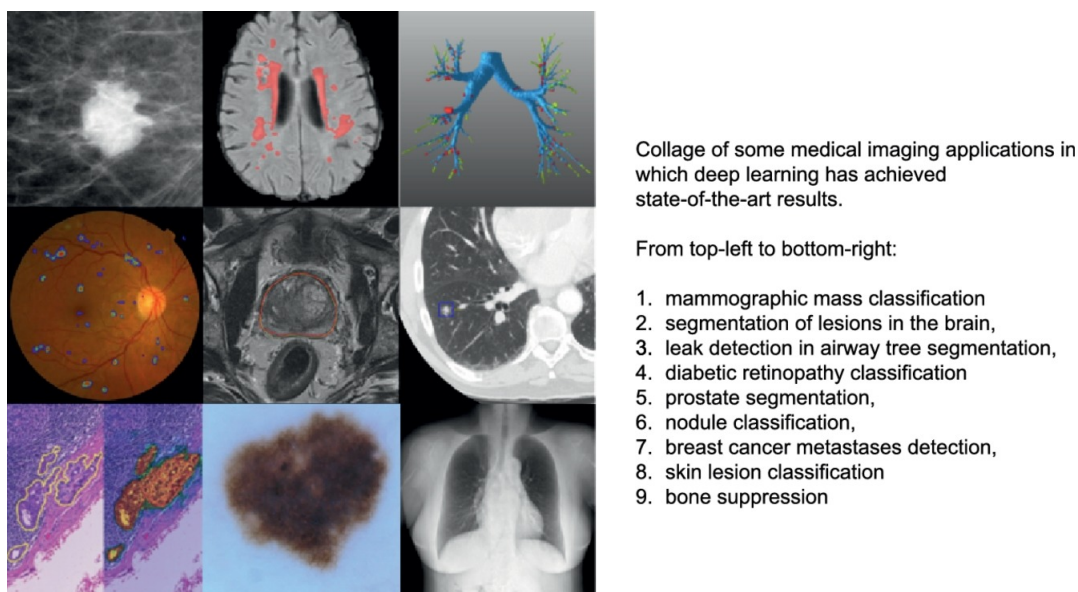


Figure 2: Various applications of medical imaging

One seminal work in the field of biomedical image segmentation is the U-Net architecture proposed by Ronneberger et al. [4]. The U-Net architecture incorporates a contracting path to capture contextual information and a symmetric expanding path for precise localization. It has been successfully applied to various segmentation tasks, including neuronal structure segmentation in electron microscopic stacks and cell tracking in transmitted light microscopy images. The U-Net demonstrated superior performance compared to prior methods, such as sliding-window convolutional networks. The architecture is efficient and capable of real-time segmentation on modern GPUs. The availability of the full implementation and pre-trained networks further facilitates its adoption in medical image analysis.

The application of deep learning techniques in radiation therapy and dose prediction is an active area of research. While deep learning has shown promising results in medical image analysis, its potential for dose prediction in radiation therapy is still being explored. By leveraging the capability of deep learning models to extract meaningful features from CT scan images, it is possible to develop accurate dose prediction models that can assist in treatment planning. The use of deep learning-based dose prediction can potentially reduce the computational complexity associated with Monte Carlo simulations and provide faster and more efficient predictions.

2.2 Computed Tomography (CT) Imaging

Computed Tomography (CT) is a widely used medical imaging technique that provides detailed cross-sectional images of the human body. It utilizes a combination of X-ray technology, advanced detectors, and sophisticated algorithms to produce high-resolution images [1].

CT scan images are created by acquiring a series of X-ray projections from multiple angles around the patient's body. These X-ray projections are then processed using mathematical algorithms to reconstruct a three-dimensional representation of the internal structures.

The basic principle of CT imaging involves the attenuation of X-rays as they pass through different tissues in the body. Different tissues have varying X-ray attenuation properties, which are influenced by factors such as density and atomic composition. These attenuation differences allow CT scanners to differentiate between various anatomical structures and visualize abnormalities or pathologies.

CT scan images are typically presented as a series of axial slices, which represent thin cross-sectional slices of the patient's body. Each slice contains a matrix of pixels, with the intensity of each pixel corresponding to the X-ray attenuation value at that particular location.

The diagnostic utility of CT imaging lies in its ability to provide detailed anatomical information with excellent spatial resolution. It has become an indispensable tool in various medical fields, including oncology, neurology, cardiology, and trauma imaging.

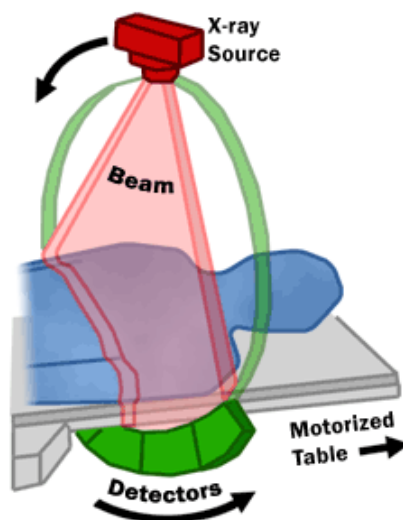


Figure 3: Drawing of CT fan beam and patient in a CT imaging system

2.3 Deep Learning and Medical Image Analysis

Deep learning, a subset of machine learning, has emerged as a powerful tool in medical image analysis. It involves training artificial neural networks with multiple layers to learn hierarchical representations from input data. These networks can automatically extract features and patterns from medical images, enabling tasks such as image classification, segmentation, and reconstruction.

The combination of deep learning and medical image analysis has shown great promise in improving diagnostic accuracy, treatment planning, and patient outcomes. Deep learning algorithms, particularly convolutional neural networks (CNNs), have achieved state-of-the-art performance in various medical imaging tasks, including tumor detection, organ segmentation, and disease classification.

By leveraging the vast amount of data available in medical imaging archives and incorporating deep learning techniques, researchers have been able to develop advanced models that can assist radiologists in making more accurate diagnoses, predicting treatment responses, and personalizing patient care.

For a deeper understanding of the potential and challenges of deep learning in medical imaging, Wang provides valuable insights in the perspective article [5].

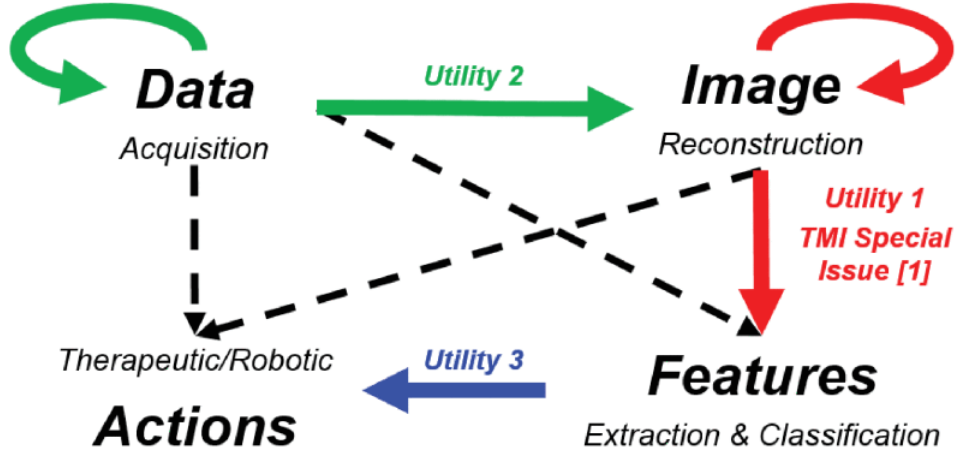


Figure 4: Big picture of deep imaging – A full fusion of medical imaging and deep learning

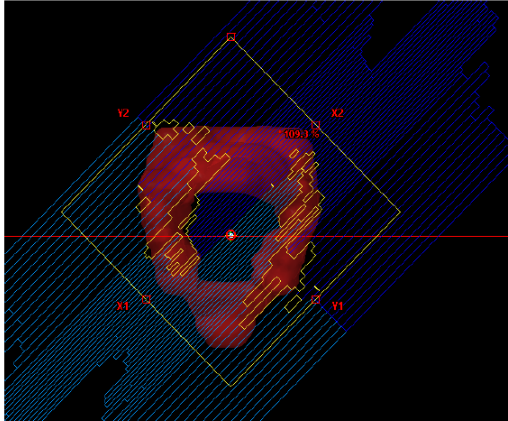
3 Data

3.1 Clinical Setup

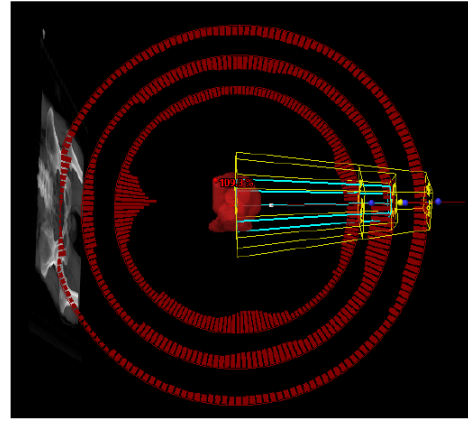
In our study, the dataset was acquired using the VMAT technique on the Clinac 2100 C machine using the following setup:

- Localization: Prostate
- Machine: Clinac 2100 C
- Technique: VMAT (Volumetric Modulated Arc Therapy)

These types of machines typically require 8-30 megavolts of energy.



Beam eye view



Visualisation of 3 arcs

Figure 5: Visualisation of the setup

VMAT:

The Volumetric Modulated Arc Therapy (VMAT) technique is a modern radiation therapy method, which enables precise delivery of radiation doses to tumors while minimizing exposure to surrounding healthy tissues. VMAT utilizes rotational arcs of the radiation beam, allowing for continuous and dynamic modulation of the beam intensity during treatment.

In VMAT, the treatment machine (Clinac 2100 C in our case) rotates around the patient in a 360-degree arc while the treatment beam is continuously adjusted in real-time. This dynamic control of the beam's intensity and shape allows for optimized dose distributions, conforming to the tumor's shape with high precision [3].

Compared to traditional techniques, such as intensity-modulated radiation therapy (IMRT), VMAT offers several advantages, including shorter treatment times, reduced radiation exposure to healthy tissues, and improved treatment accuracy. These benefits make VMAT an increasingly popular choice in modern radiation therapy for cancer treatment, particularly in cases where precise dose delivery and sparing of critical structures are crucial, as in prostate cancer treatment.

the use of the VMAT technique is to ensure precise and efficient delivery of radiation doses to the prostate cancer patients while minimizing potential side effects.

3.2 Dataset

The dataset consists of eight patients, each with a set of CT scans in DICOM (.dcm) format and corresponding fluence data in MetaImage (.mhd) format. The CT scans provide detailed cross-sectional images of the patient's anatomy, while the fluence data captures the spatial distribution of the radiation beam intensity delivered during treatment. Each treatment arc consists of 178 radiation beams, resulting in a total of 178 fluence maps for each patient. These fluence maps, provide valuable information about the radiation dosage delivered to different regions within the patient's prostate.

It is important to note that all patient data used in this study were anonymized to ensure privacy and comply with ethical considerations.

3.3 Data Preprocessing ¹

The preprocessing step involved converting the raw data, obtained from DICOM and MHD files, into binary images suitable for further analysis. Two provided scripts, `dicom.py` and `gate.py`, were utilized for this purpose.

3.3.1 CT Scan Preprocessing

In the case of DICOM data, the `dicom.py` script was used. It employed the `pydicom` library to read the DICOM files and extract relevant information. The script focused on Multi-Leaf Collimator (MLC) analysis by extracting MLC properties such as leaf widths and positions from the DICOM files. It constructed an MLC image by mapping the leaf positions onto a 2D matrix.

To make the image suitable for further analysis, modifications were made to convert it into a binary image and resize it to 320x320 pixels. The resulting binary MLC image was saved as a PNG file.

Figure 6 shows the output of this script.

3.3.2 Fluence Data Preprocessing

For the GATE simulation data, the `gate.py` script was employed. It processed the fluence data obtained from Monte Carlo simulations using the GATE software. The script used the `SimpleITK` library to read the fluence data stored in MHD files. It extracted the fluence matrix from the MHD files and performed necessary manipulations such as rotation, normalization, and conversion into a binary image and resizing to 320x320 pixels.

Figure 7 shows the output of this script.

¹check the appendix A.1 for futher description

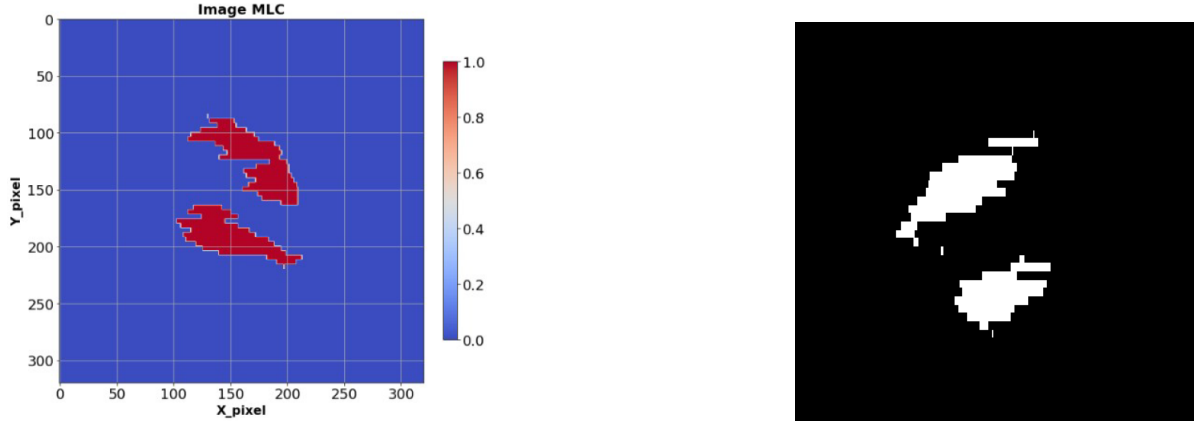


Figure 6: left: the output of the dicom.py script, an MLC image. right: the modified binary image saved. (These 2 images are not for the same patient)

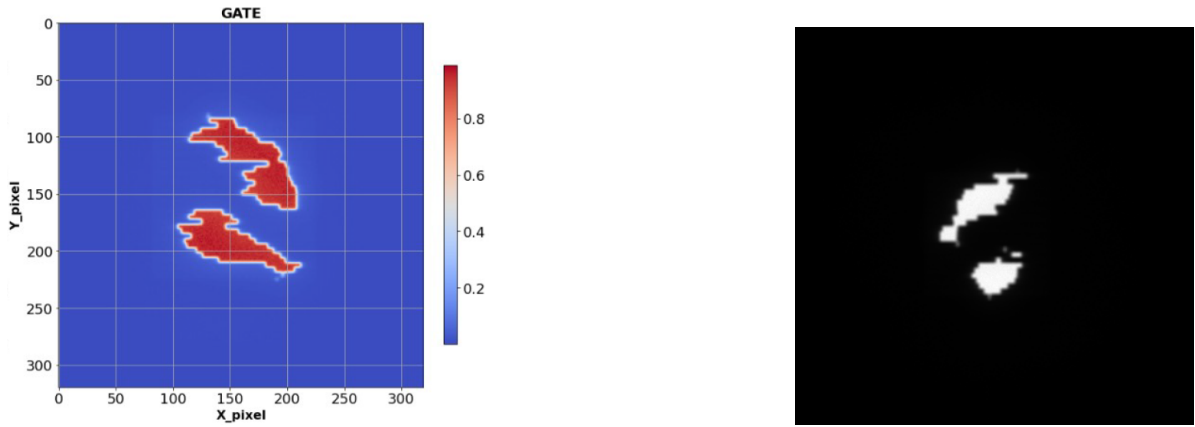


Figure 7: left: the output of the gate.py script, an gate image. right: the modified binary image saved. (These 2 images are not for the same patient)

To facilitate the training and evaluation of the deep learning model, the CT scan images were normalized to have zero mean and unit variance to alleviate any intensity variations across different patients. Subsequently, the images were reshaped to a binary representation, where the background pixels were set to 0 and the region of interest (ROI) pixels were set to 1, and resized to 320x320 pixels. This binary representation allows the U-Net model to focus on accurately predicting the dose distribution within the ROI. The dataset was also divided into training and testing subsets. A random split of 80% for training and 20% for testing was employed. These steps lay the foundation for training the deep learning model to predict the dose distribution accurately.

4 Model Training and Evaluation

4.1 UNet Architecture

The UNet architecture, proposed by Ronneberger et al. [4], is a convolutional neural network specifically designed for biomedical image segmentation tasks. It has been widely adopted in the field due to its effectiveness in capturing both local and global information.

The architecture of UNet consists of an encoding path (contracting path) and a decoding path (expanding path), which allows for multi-level feature extraction and precise localization of segmentation boundaries. The contracting path follows a typical convolutional neural network structure, consisting of repeated convolutional layers followed by max-pooling operations. This path aims to capture context and create a feature hierarchy by progressively reducing the spatial dimensions while increasing the number of feature channels.

The expanding path mirrors the contracting path and is designed to enable precise localization. It consists of repeated upsampling layers followed by convolutional layers. Skip connections are established between corresponding layers in the contracting and expanding paths, allowing the flow of low-level and high-level features. These skip connections facilitate the combination of coarse-grained, high-level semantic information from the contracting path with fine-grained, low-level spatial information from the expanding path.

The UNet architecture emphasizes the use of data augmentation to improve training efficiency, particularly in cases where only a limited number of annotated samples are available. By applying various augmentations, such as rotations, translations, and elastic deformations, the network can effectively learn from a small number of training images.

The U-Net model we implemented follows the UNet architecture described above. The contracting path consists of convolutional layers with ReLU activation followed by max-pooling operations, while the expanding path comprises upsampling layers followed by convolutional layers. Skip connections are established to combine high-level semantic information from the contracting path with low-level spatial information from the expanding path. The final layer of the U-Net model uses a 1×1 convolution followed by a sigmoid activation function to generate the segmentation mask. (Check the Appendix B.2 for detailed description of the model)

Figure 8 illustrates the UNet architecture as presented in the original paper by Ronneberger et al.

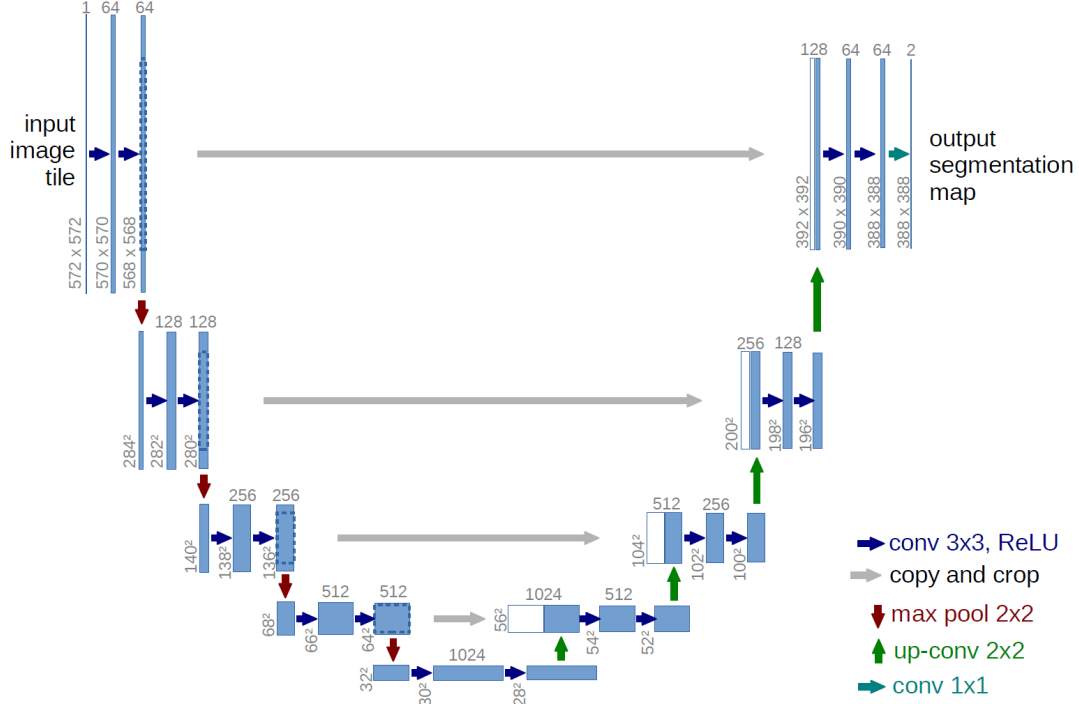


Figure 8: The UNet architecture [4].

4.2 Training

The training process aims to optimize the U-Net model using the prepared dataset. In this subsection, we provide an overview of the training procedure and include a code snippet for reference.

4.2.1 Dataset Preparation

Before delving into the training process, it is essential to ensure that the dataset is properly organized and accessible. The `dataset` folder, provided with this project, contains the preprocessed data in the appropriate format. the `data_utils.py` is responsible for loading the images, resizing them to a consistent size of 320×320 pixels, converting them to grayscale, normalizing the pixel values to the range $[0, 1]$, and splitting the dataset into training and validation sets. The input and output data are stored in numpy arrays, and their shapes are printed to verify that the data preprocessing step is successful.

The `train.py` script assumes that the training and validation data are already split and available as `x_train`, `x_val`, `y_train`, and `y_val` arrays in the `data_utils.py` file.

4.2.2 Model Compilation

Once the dataset is prepared, the U-Net model is compiled with the Adam optimizer and the mean squared error (MSE) loss function. The Adam optimizer is an efficient optimization algorithm that adapts the learning rate dynamically during training, while MSE is commonly used for regression tasks. Additionally, we monitor the mean absolute error (MAE) metric during training to assess the model's performance.

The code snippet below demonstrates the compilation of the U-Net model:

```
# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error', metrics=['mae'])
```

4.2.3 Model Training

The model is trained using the prepared dataset, specifying the batch size and the number of epochs. In this case, a batch size of 8 and 10 epochs are used. The training process involves feeding batches of

training data to the model and updating the model's parameters based on the optimization algorithm. The validation data is used to evaluate the model's performance after each epoch. The following code snippet illustrates the training process:

```
# Train the model with callbacks
batch_size = 8
epochs = 10

model.fit(x_train, y_train, batch_size=batch_size,
          epochs=epochs, validation_data=(x_val, y_val),
          callbacks=callbacks)
```

The average running time for training the model is a few minutes on a modern GPU, although the actual time may vary depending on the hardware configuration.

4.2.4 Model Evaluation

After the training process is complete, the trained model's performance is evaluated on the validation set. The evaluation involves computing the loss and accuracy metrics to assess how well the model generalizes to unseen data. The model's ability to accurately reconstruct the input images is of particular interest in this task.

The following code snippet demonstrates the evaluation process:

```
# Evaluate the model
loss, accuracy = model.evaluate(x_val, y_val)
print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy)
```

The performance of the image reconstruction model is assessed using the following evaluation metrics:

- **Validation Loss:** The validation loss measures the discrepancy between the predicted outputs and the actual ground truth values during the validation phase. A lower validation loss indicates better performance. In our experiments, the obtained validation loss was 0.0283.
- **Validation Accuracy:** The validation accuracy is the proportion of correctly predicted instances out of the total number of instances in the validation set. It provides an overall measure of how well the model is classifying or predicting the target variable. For our model, the validation accuracy was found to be 0.0378.

These evaluation metrics provide insights into the performance of the model and serve as quantitative measures to assess the accuracy and quality of the image reconstruction results.

4.2.5 Saving Images

To visualize the performance of the model, we save the input and reconstructed images from the validation set. The input images represent the original images, while the reconstructed images are the output of the U-Net model after processing the input images.

The code snippet below demonstrates the process of saving the input and reconstructed images:

```
# Create directories to save the images
input_images_dir = paths.input_images_dir
reconstructed_images_dir = paths.reconstructed_images_dir

os.makedirs(input_images_dir, exist_ok=True)
os.makedirs(reconstructed_images_dir, exist_ok=True)

Save the input and reconstructed images
for i in range(len(x_val)):
```

```
# Get the input image
input_image = (x_val[i] * 255).astype(np.uint8)
input_image_path = os.path.join(input_images_dir, f'input_{i+1}.png')
Image.fromarray(input_image.squeeze()).save(input_image_path)

# Get the reconstructed image
reconstructed_image = model.predict(np.expand_dims(x_val[i], axis=0))
reconstructed_image = (reconstructed_image[0] * 255).astype(np.uint8)
reconstructed_image_path = os.path.join(reconstructed_images_dir, f'reconstructed_{i+1}.png')
Image.fromarray(reconstructed_image.squeeze()).save(reconstructed_image_path)
```

4.3 Model Output and Logs

The trained model generates two important outputs: the `model.h5` file and the `logs` directory. The `model.h5` file is a serialized representation of the trained model. It contains the learned weights and architecture of the model, allowing it to be loaded and used for various purposes. This file can be utilized for further analysis, inference, or deployment in other applications. To load the model, one can use the appropriate libraries or frameworks in his preferred programming language and load the model using the `load_model()` function.

The `logs` directory contains valuable information about the training process. It typically includes logs such as training loss, validation loss, and other metrics collected during the training phase. These logs provide insights into the model's performance, convergence, and any potential issues during training. They can be visualized using tools like TensorBoard or plotted using libraries like Matplotlib. Analyzing the logs can help in understanding the model's behavior, identifying areas for improvement, and making informed decisions regarding hyperparameter tuning or model modifications.

In theory, the `model.h5` file and the `logs` directory can be used as follows:

- **Model Deployment:** The `model.h5` file can be deployed in production environments to perform inference tasks. It can be integrated into software systems or applications to make predictions on new or unseen data. By loading the trained model, you can utilize its learned capabilities to make accurate predictions or perform image reconstruction tasks on real-world data.
- **Model Evaluation:** The `model.h5` file and the `logs` directory are valuable resources for model evaluation. You can use them to assess the model's performance on a separate test dataset, calculate additional evaluation metrics, and compare the results with other models or baselines. This evaluation process helps in understanding the model's generalization capabilities and its suitability for the intended application.
- **Further Development:** The `model.h5` file serves as a starting point for further model development. It can be fine-tuned or used as a base model for transfer learning. Additionally, the logs can guide the refinement of the model architecture, hyperparameter tuning, or the selection of different optimization strategies.

The `model.h5` file and the `logs` directory provide valuable resources that enable the utilization, evaluation, and improvement of the trained model in various contexts and scenarios.

5 Results

5.1 Training and Evaluation

We trained the U-Net model on a dataset of 2492 images. The dataset was split into training and validation sets with a ratio of 80:20. The model was trained for 10 epochs using a batch size of 8. We used the Adam optimizer.

During the training process, we monitored the loss and mean absolute error (MAE) on both the training and validation sets. However, the results obtained were not satisfactory. The model struggled to learn meaningful representations from the input data, resulting in reconstructed images that appeared as black

images.

The validation loss and MAE achieved at the end of the training were 0.0283 and 0.0378, respectively. These metrics, specifically the MAE which shows only 3.78% accuracy, indicate poor performance in accurately reconstructing the does distribution images from the CT-scan inputs.

The following snippet represents the terminal output

```
Epoch 1/10
 1/250 [.....] - ETA: 1:00:42 - loss: 0.2437 - mae: 0.4923
250/250 [=====] - 4981s 20s/step - loss: 0.0306 - mae: 0.0437 -
val_loss: 0.0283 - val_mae: 0.0378

Epoch 2/10
 1/250 [.....] - ETA: 1:41:07 - loss: 0.0235 - mae: 0.
250/250 [=====] - 5041s 20s/step - loss: 0.0275 - mae: 0.0368 -
val_loss: 0.0283 - val_mae: 0.0378

Epoch 3/10
 1/250 [.....] - ETA: 45:50 - loss: 0.0277 - mae: 0.03
250/250 [=====] - 5132s 21s/step - loss: 0.0275 - mae: 0.0368 -
val_loss: 0.0283 - val_mae: 0.0378

Epoch 4/10
 1/250 [.....] - ETA: 1:47:34 - loss: 0.0353 - mae: 0.
250/250 [=====] - 5087s 20s/step - loss: 0.0275 - mae: 0.0368 -
val_loss: 0.0283 - val_mae: 0.0378

Epoch 5/10
 1/250 [.....] - ETA: 47:01 - loss: 0.0200 - mae: 0.02
250/250 [=====] - 5043s 20s/step - loss: 0.0275 - mae: 0.0368 -
val_loss: 0.0283 - val_mae: 0.0378

Epoch 6/10
 1/250 [.....] - ETA: 1:44:47 - loss: 0.0244 - mae: 0.
250/250 [=====] - 5104s 20s/step - loss: 0.0275 - mae: 0.0368 -
val_loss: 0.0283 - val_mae: 0.0378

Epoch 7/10
 1/250 [.....] - ETA: 47:33 - loss: 0.0266 - mae: 0.03
250/250 [=====] - 5063s 20s/step - loss: 0.0275 - mae: 0.0368 -
val_loss: 0.0283 - val_mae: 0.0378

Epoch 8/10
 1/250 [.....] - ETA: 1:38:57 - loss: 0.0340 - mae: 0.
250/250 [=====] - 5086s 20s/step - loss: 0.0275 - mae: 0.0368 -
val_loss: 0.0283 - val_mae: 0.0378

Epoch 9/10
 1/250 [.....] - ETA: 46:40 - loss: 0.0203 - mae: 0.02
250/250 [=====] - 5119s 21s/step - loss: 0.0275 - mae: 0.0368 -
val_loss: 0.0283 - val_mae: 0.0378

Epoch 10/10
 1/250 [.....] - ETA: 45:50 - loss: 0.0348 - mae: 0.04
250/250 [=====] - 4999s 20s/step - loss: 0.0275 - mae: 0.0368 -
val_loss: 0.0283 - val_mae: 0.0378

Validation Loss: 0.028336159884929657 Validation Accuracy: 0.03778257966041565
```

5.2 Reconstructed Images

Unfortunately, the reconstructed images generated by the trained model did not exhibit the desired level of quality. Instead, the reconstructed images appeared as black images with no discernible features or details.



Figure 9: reconstructed image

5.3 Terminal Output

During the training process, we encountered several warning and error messages related to TensorFlow and CUDA when using the CompuPhys server. These messages indicated issues with the CUDA configuration and the absence of a CUDA-capable device. As a result, the model was not able to utilize the GPU for training, which may have impacted the training process and the quality of the reconstructed images.

The following snippet showcase these errors:

```
afadlal2@compuphys-calc:~/code$ python3 unet_v4.py
2023-06-21 12:34:39.234070: W tensorflow/stream_executor/platform/default/dso_loader.cc:64]
Could not load dynamic library 'libcudart.so.11.0'; dLError: libcudart.so.11.0:
cannot open shared object file: No such file or directory;
LD_LIBRARY_PATH: /opt/gildas-exe-nov20a/x86_64-ubuntu18.04-gfortran/lib:
2023-06-21 12:34:39.234116: I tensorflow/stream_executor/cuda/cudart_stub.cc:29]
Ignore above cudart dLError if you do not have a GPU set up on your machine.
Training set - Input shape: (1993, 320, 320, 1) Output shape: (1993, 320, 320, 1)
Validation set - Input shape: (499, 320, 320, 1) Output shape: (499, 320, 320, 1)
2023-06-21 12:34:51.459707: E tensorflow/stream_executor/cuda/cuda_driver.cc:271]
failed call to cuInit: CUDA_ERROR_NO_DEVICE: no CUDA-capable device is detected
```

5.4 Output Analysis

The model outputs consist of the trained model.h5 file and the logs directory.

The model.h5 file represents the trained U-Net model that has learned from the training data. It can be loaded and used for various purposes, such as performing inference on new CT scan images or serving as a starting point for further model development.

The logs directory contains valuable information about the training process and model performance, including training and validation loss curves, accuracy metrics, and other relevant statistics. This information can be used for performance analysis, model selection, and fine-tuning purposes.

Furthermore, if the reconstructed images were of good quality and accurately captured the dose distribution, the gate.py script could be employed. This script can potentially be used to reverse the binary image and estimate the fluence parameters that govern the dose distribution. This shows a potential to derive the correct parameters for radiation therapy planning, ultimately contributing to improved treatment outcomes and patient care.

6 Discussion

6.1 Analysis of Results

The results of the image reconstruction using the U-Net model are not as expected, with poor quality and inaccurate reconstructions. The reconstructed images are plain black images. This indicates that the model was unable to effectively capture and reconstruct the underlying image features.

Several factors could have contributed to the poor performance of the U-Net model:

1. Inadequate Model Capacity

The capacity of the U-Net model may also have played a role in the poor results. The current model architecture consists of relatively fewer layers and parameters compared to more advanced variations of U-Net used in state-of-the-art image reconstruction tasks. This limited capacity might have restricted the model's ability to learn intricate image details, resulting in less accurate reconstructions.

2. Suboptimal Hyperparameters

The choice of hyperparameters, such as batch size, and epochs, can significantly impact the model's performance. It is possible that suboptimal hyperparameter settings were used during training, leading to slow convergence or model instability. Fine-tuning these hyperparameters could potentially improve the model's performance.

6.2 Potential Solutions

To address the limitations identified, the following strategies could be explored:

1. Architecture Modifications

Considering the limited capacity of the current U-Net model, modifications to the architecture can be considered. Increasing the depth of the network, adding more convolutional layers, or incorporating skip connections from lower-level feature maps could improve the model's ability to capture fine details and improve the reconstruction quality.

2. Hyperparameter Tuning

A thorough exploration of hyperparameter settings should be conducted to identify optimal configurations. Techniques such as grid search or random search can be used to find the best combination of learning rate, batch size, regularization parameters, and optimizer settings. Hyperparameter tuning could potentially lead to improved convergence, stability, and overall performance.

7 Challenges

During the course of this project, I faced the following challenges:

- **Limited exposure to medical physics and machine learning:** This project marked my first experience in the field of medical physics, requiring additional effort to understand the fundamental concepts. It was also my first time working with machine learning, necessitating a learning curve to grasp the necessary skills.
- **Technical difficulties accessing the university server and lack of GPU:** I faced challenges in accessing the university server from my laptop, which meant I had to go to the lab each time I needed to run my scripts. Additionally, the Compuphys server used for the project lacked a GPU, resulting in significantly longer training times for the U-Net model. Each training run took approximately 16-20 hours, extending the project duration and requiring careful planning.

8 Conclusion

In conclusion, this research project aimed to explore the potential of deep learning techniques in image reconstruction for CT scans with external radiation exposure, with a specific focus on dose prediction for prostate cancer patients.

Although the obtained results did not meet the desired accuracy and image quality, valuable insights were gained regarding the challenges and considerations involved in constructing an effective deep learning model for dose prediction. The limitations identified, such as suboptimal model architecture and parameter choices, highlight areas for further investigation and improvement. Future research should focus on refining the model's architecture, exploring alternative parameter configurations, and optimizing the training process to enhance the accuracy and quality of dose prediction in radiation therapy.

References

- [1] J.T. Bushberg, J.A. Seibert, E.M. Leidholdt Jr, and J.M. Boone. *The Essential Physics of Medical Imaging, Third Edition*. Lippincott Williams & Wilkins, 2011.
- [2] Geert Litjens, Thijs Kooi, Babak Ehteshami Bejnordi, Arnaud Arindra Adiyoso Setio, Francesco Ciompi, Mohsen Ghafoorian, Jeroen A.W.M. van der Laak, Bram van Ginneken, and Clara I. Sánchez. A survey on deep learning in medical image analysis. *Medical Image Analysis*, 42: 60–88, 2017. ISSN 1361-8415. doi:<https://doi.org/10.1016/j.media.2017.07.005>. URL <https://www.sciencedirect.com/science/article/pii/S1361841517301135>.
- [3] Karl Otto. Volumetric modulated arc therapy: Imrt in a single gantry arc. *Medical Physics*, 35(1): 310–317, 2008. doi:10.1118/1.2818738. URL <https://aapm.onlinelibrary.wiley.com/doi/abs/10.1118/1.2818738>.
- [4] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pages 234–241, 2015. URL https://link.springer.com/chapter/10.1007/978-3-319-24574-4_28.
- [5] Ge Wang. A perspective on deep imaging. *IEEE Access*, 4:8914–8924, 2016. doi:10.1109/ACCESS.2016.2624938.

A Appendix

A.1 Preprocessing Method for Raw Data

The preprocessing step involved converting the DICOM and MHD files into binary images suitable for further analysis. The provided scripts, `dicom.py` and `gate.py`, facilitated this conversion by utilizing various libraries and custom modules.

DICOM Processing

The `dicom.py` script was used to handle DICOM files, which contain medical imaging data. The following steps were performed during the DICOM preprocessing:

1. **DICOM Reading:** The script utilized the `pydicom` library to read DICOM files from the specified folder. It provided a method, `get_file`, which allowed the retrieval of specific DICOM files based on their types, such as CT, RTPlan, or RTStructure.
2. **Multi-Leaf Collimator (MLC) Analysis:** The `MLC` class in `dicom.py` focused on analyzing the Multi-Leaf Collimator (MLC) properties. It extracted information such as leaf widths and positions from the DICOM files. Specifically, the `get_sequence_from_dicom` method was used to retrieve the leaf sequence for a given control point, arc, and treatment phase.
3. **MLC Image Construction:** The `MLC` class further facilitated the construction of an MLC image by mapping the extracted leaf positions onto a 2D matrix. The resulting image initially represented the MLC as a grayscale image. However, adjustments were made to convert it into a binary image format, which is more suitable for subsequent analysis. The modified script saved the resulting binary MLC image as a PNG file.
4. **Gantry Angle Extraction:** The `Gantry` class in `dicom.py` handled the extraction of gantry angles for a given arc and treatment phase. It provided the `get_angle_sequence` method, which retrieved the gantry angles from the DICOM files.

GATE Simulation Data Processing

The `gate.py` script was specifically designed to process data obtained from Monte Carlo simulations using the GATE (Geant4 Application for Tomographic Emission) software. The following steps were performed during the GATE simulation data preprocessing:

1. **Fluence Matrix Extraction:** The `Fluence` class in `gate.py` was responsible for extracting the fluence matrix from the GATE simulation results. It read the fluence data stored in MHD (MetaIO) files using the `SimpleITK` library. The `get_fluence` method retrieved the fluence matrix and returned it as a 2D numpy array.
2. **Fluence Matrix Manipulation:** Once the fluence matrix was obtained, several transformations were applied. Firstly, the matrix was rotated by 90 degrees using the `rotate` method from the `Operation` module. This rotation ensured that the matrix alignment matched the MLC image and other data. Additionally, further adjustments were made to normalize the matrix and convert it into a binary image format suitable for subsequent analysis.

Important Notice

Please note that the processed dataset, is included with this report in the ‘dataset’ folder. The data has been preprocessed using the ‘dicom.py’ and ‘gate.py’ scripts, but the scripts themselves are not included as these are not my scripts.

B Technical Report

In this part, we provide a technical report of the code used for implementing the U-Net model for image segmentation. The code is implemented using TensorFlow and Keras, and the provided report outlines the key components and functionality of the code.

B.1 Code Structure

The code is organized into several modules and files, each serving a specific purpose. Here is an overview of the code structure:

- **unet.py**: This file contains the implementation of the U-Net model. It defines the architecture of the model, including the contracting and expanding paths, skip connections, and the final segmentation layer.
- **data_utils.py**: This file contains utility functions for data preprocessing and loading. It includes functions for resizing images, loading datasets, and splitting data into training and validation sets.
- **train.py**: This file implements the training pipeline for the U-Net model. It includes functions for compiling the model, defining loss functions and evaluation metrics, and running the training loop.
- **paths.py**: This file defines the file paths and directories used by the code, such as the model checkpoint path, log directory, input and reconstructed image directories, etc.

B.2 Model Definition

In this section, we provide a detailed description of the U-Net model 'unet.py' architecture used for image reconstruction.

The U-Net model is implemented using the TensorFlow-Keras framework. The model consists of two main parts: the contracting path and the expansive path. The contracting path captures the context and reduces the spatial dimensions of the input image, while the expansive path recovers the spatial information and generates the reconstructed image.

The model is defined using the following code:

```
import numpy as np
from tensorflow.keras import layers, models

def unet(input_shape):
    inputs = layers.Input(shape=input_shape)

    # Contracting Path
    conv1 = layers.Conv2D(64, 3, activation='relu', padding='same')(inputs)
    conv1 = layers.Conv2D(64, 3, activation='relu', padding='same')(conv1)
    pool1 = layers.MaxPooling2D(pool_size=(2, 2))(conv1)

    conv2 = layers.Conv2D(128, 3, activation='relu', padding='same')(pool1)
    conv2 = layers.Conv2D(128, 3, activation='relu', padding='same')(conv2)
    pool2 = layers.MaxPooling2D(pool_size=(2, 2))(conv2)

    conv3 = layers.Conv2D(256, 3, activation='relu', padding='same')(pool2)
    conv3 = layers.Conv2D(256, 3, activation='relu', padding='same')(conv3)
    pool3 = layers.MaxPooling2D(pool_size=(2, 2))(conv3)

    conv4 = layers.Conv2D(512, 3, activation='relu', padding='same')(pool3)
    conv4 = layers.Conv2D(512, 3, activation='relu', padding='same')(conv4)
    drop4 = layers.Dropout(0.5)(conv4)
    pool4 = layers.MaxPooling2D(pool_size=(2, 2))(drop4)
```

```

conv5 = layers.Conv2D(1024, 3, activation='relu', padding='same')(pool4)
conv5 = layers.Conv2D(1024, 3, activation='relu', padding='same')(conv5)
drop5 = layers.Dropout(0.5)(conv5)

# Expansive Path
up6 = layers.Conv2D(512, 2, activation='relu', padding='same')
(layers.UpSampling2D(size=(2, 2))(drop5))
merge6 = layers.concatenate([drop4, up6], axis=3)
conv6 = layers.Conv2D(512, 3, activation='relu', padding='same')(merge6)
conv6 = layers.Conv2D(512, 3, activation='relu', padding='same')(conv6)

up7 = layers.Conv2D(256, 2, activation='relu', padding='same')
(layers.UpSampling2D(size=(2, 2))(conv6))
merge7 = layers.concatenate([conv3, up7], axis=3)
conv7 = layers.Conv2D(256, 3, activation='relu', padding='same')(merge7)
conv7 = layers.Conv2D(256, 3, activation='relu', padding='same')(conv7)

up8 = layers.Conv2D(128, 2, activation='relu', padding='same')
(layers.UpSampling2D(size=(2, 2))(conv7))
merge8 = layers.concatenate([conv2, up8], axis=3)
conv8 = layers.Conv2D(128, 3, activation='relu', padding='same')(merge8)
conv8 = layers.Conv2D(128, 3, activation='relu', padding='same')(conv8)

up9 = layers.Conv2D(64, 2, activation='relu', padding='same')
(layers.UpSampling2D(size=(2, 2))(conv8))
merge9 = layers.concatenate([conv1, up9], axis=3)
conv9 = layers.Conv2D(64, 3, activation='relu', padding='same')(merge9)
conv9 = layers.Conv2D(64, 3, activation='relu', padding='same')(conv9)

outputs = layers.Conv2D(1, 1, activation='sigmoid')(conv9)

model = models.Model(inputs=inputs, outputs=outputs)

return model

```

Contracting Path

The contracting path consists of multiple convolutional layers with a 3x3 kernel size and 'relu' activation. After each convolutional layer, another 3x3 convolutional layer is applied. The contracting path also includes max pooling layers with a pool size of 2x2 to reduce the spatial dimensions.

Expansive Path

The expansive path performs upsampling and concatenation operations to recover the spatial information lost during the contracting path. Each upsampling layer doubles the spatial dimensions of the input. The upsampled feature maps are then concatenated with the corresponding feature maps from the contracting path. After each concatenation, two 3x3 convolutional layers with 'relu' activation are applied.

Output

The final layer of the U-Net model is a 1x1 convolutional layer with a 'sigmoid' activation function. It generates the reconstructed image with a single channel.

This U-Net architecture has been widely used for various image segmentation and image-to-image translation tasks. However, in our experiments, this particular implementation did not yield satisfactory results for image reconstruction.

B.3 Modules

The code has the following Modules:

- TensorFlow: Version 2.0 or higher.
- Keras: Version 2.2.0 or higher.
- NumPy: Version 1.18.0 or higher.
- PIL: Version 8.2.0 or higher (for saving images).

Make sure to install these modules before running the code.

B.4 Usage

To use the code, follow these steps:

1. Prepare the dataset: Ensure that the dataset is properly organized and accessible. The provided **dataset** folder contains the necessary data in the correct format. Make sure the dataset folder is placed in the same directory as the code files. The code assumes that the training and validation data are already split and available as **x_train**, **x_val**, **y_train**, and **y_val** arrays in the **data.utils.py** file.
2. Configure the paths: Adjust the file paths and directories in the **paths.py** file according to your system and requirements.
3. Train the model: Run the **train.py** file to start the training process. The code will load the dataset, initialize the model, and train it using the specified parameters. The training progress and performance metrics will be displayed during training. The trained model will be saved in the specified model checkpoint path.
4. Evaluate the model: After training, you can evaluate the performance of the trained model by calculating evaluation metrics on the validation set (this is done automatically). The evaluation metrics can be obtained by running appropriate code or functions in the **train.py** file.
5. Save input and reconstructed images: The code includes functionality to save the input and reconstructed images from the validation set. The images will be saved in the directories specified in the **paths.py** file.
6. Training and Inference: The training process involves optimizing the U-Net model using the provided dataset. The code assumes that the dataset has been properly prepared and split into training and validation sets. The average running time for training is a few minutes on a modern GPU, but it may vary depending on the hardware configuration.

During training, the model is compiled with the Adam optimizer and the mean squared error loss function. The training is performed for a specified number of epochs, with a batch size of 8. The model is evaluated on the validation set after training, and the loss and accuracy metrics are reported.

After training, the model is used for inference on the validation set. The input images and their corresponding reconstructed images are saved in separate directories for further analysis. Please ensure that the necessary directories are created or modified in the code to save the images correctly.